

# 異種書誌 LOD への汎用 RDF → グラフ変換と R-GCN ノード分類： クラス分離度の横断比較

日野岡雅人  
202621778

s2621778@U.TSUKUBA.AC.JP

## 1. 目的

本レポートの目的は、スキーマ・言語・ドメインが大きく異なる実世界の書誌・典拠 Linked Open Data (LOD) に対して、単一の RDF → グラフ変換パイプラインを設定ファイルの差し替えだけで適用し、同一の R-GCN (Relational Graph Convolutional Network) アーキテクチャでノード分類を行い、学習された埋め込み空間におけるクラス分離度をエンドポイント横断で比較することである。

ここで本レポートの新規性を明確にする。まず、モデルである R-GCN (Schlichtkrull et al., 2018) には、モデルとしての新規性は一切ない。R-GCN は RDF ナレッジグラフのノード分類における最も標準的な手法であり、原論文自体が AIFB・MUTAG・BGS・AM という RDF ベンチマークで評価している。したがって「R-GCN を LOD に応用した」こと自体を新規性として主張はしない。

そこで本レポートが主張する新規性は、モデル側ではなく**評価・分析側**に限定する。すなわち、(1) スキーマも言語もドメインも異なる複数の実世界 LOD を、エンドポイント固有のコードを一切書かずに設定ファイルだけで統一的に扱えるパイプラインを構成したこと、(2) その上で「クラスの分離しやすさ」をクラスセントロイド間コサイン類似度という単一の要約統計量で定量化し、エンドポイント横断で比較する分析軸を提示したこと、の 2 点である。

## 2. 関連研究

本レポートが採用する R-GCN (Schlichtkrull et al., 2018) は、リレーションタイプごとに重み行列を持つグラフ畳み込みを用いて、多関係グラフ (RDF) 上のノード分類やリンク予測を行う手法である。関係数が多い場合のパラメータ爆発を基底分解 (basis decomposition) で抑える点が特徴である。本レポートはこのモデルをそのまま用いる。前述のとおり、R-GCN の RDF ノード分類への適用は原論文がすでに標準的なベンチマークで行っている。

RDF からグラフ機械学習用のデータセットを生成するという観点で本実験に最も近い先行研究は、AutoRDF2GML (Färber et al., 2024) である。これは RDF から異種グラフ機械学習用データセットを半自動生成する枠組みであり、オントロジーに基づく特徴量抽出やエンティティ間関係の変換を扱う。本レポートは、これを性能比較の対象としてではなく、「本実験が断念したこと」を説明する参照として位置づける。具体的には、本実験はオントロジーアライメントやエンティティリンク (owl:sameAs の解決など) を明示的にスコープ外とし、各エンドポイントを独立したグラフとして扱う点で、AutoRDF2GML よりも大幅に簡略としている。

Table 1: 対象とした 2 エンドポイント。NDL は典拠ノード自体を、DBLP は書誌レコードを分類対象とする。

| 項目      | NDL Web NDL Authorities | DBLP             |
|---------|-------------------------|------------------|
| 内容      | 日本語 主題・人物典拠             | 計算機科学文献 書誌       |
| 分類対象クラス | skos:Concept            | dblp:Publication |
| ラベル述語   | skos:inScheme           | dblp:bibtexType  |
| 言語      | 日本語                     | 英語               |

### 3. 方法

#### 3.1 対象エンドポイントとスコープ

当初は書誌・典拠系の 4 エンドポイント (DBLP・NDL Web NDL Authorities・ICCU SBN・Cervantes Virtual) を対象に計画したが、提出期限の制約により、設定の確定作業まで完了していた次の 2 件に縮小した。両者を表 1 に示す。NDL は書誌ノードを持たず、分類対象が典拠ノード自体である点で、書誌レコードを分類する DBLP とスキーマ概念が根本的に異なる。この違いを、汎用パイプラインが設定変更のみで異なるスキーマに対応できることの実証を行う。

分類対象クラスとラベル述語の URI は次のような手順で決定した。(1) `?s a ?class` のインスタンス数集計によるクラス発見、(2) 述語プロファイル (被覆率・値の型・カーディナリティ)、(3) ラベル候補述語の GROUP BY 全数集計による分布検証、の 3 段階を経て決定した。特に (3) では、少数のサンプルによる述語プロファイルが物理格納順に強く相関した非ランダムな分布を返すことを実証した。例えば DBLP の `bibtexType` を 200 件サンプルすると Incollection が 94% を占めるが、全数集計による真の分布は Article 50.7%・Inproceedings 45.4% であり、両者は全く別物であった。この事実から、サンプル統計を分布推定に用いてはならないと結論し、ラベルの確定は全数集計に基づいて行った。

#### 3.2 設定ファイル駆動 (yaml) のパイプライン

パイプラインは、SPARQL 抽出、ラベル抽出、グラフ変換、モデル学習、評価の 5 段からなる。エンドポイント固有の値 (対象クラス・ラベル述語・規模上限・特徴量に使う述語など) はすべて `configs/*.yaml` に外出しし、Python コード側には if 分岐を一切置かない。したがって新しいエンドポイントへの対応は設定ファイルの差し替えのみで完結する。本実験では、この主張を「同一コードのまま設定だけを差し替えて NDL と DBLP の双方でパイプラインが通ること」で実地に検証した。

#### 3.3 ラベルの層化抽出

分類対象クラスのインスタンスを素朴に `LIMIT max_nodes` で取得すると、物理的な格納順によるバイアスがそのまま抽出結果に混入し、規模を 1 万 5 千件に増やしても偏りは解消されない。これを避けるため、全数集計で得た上位 `top_k` 個のラベル値ごとの真の件数比に `max_nodes` をサンプリングし、ラベル値ごとに個別のクエリで抽出する層化抽出を採用した。これにより層間 (ラベル間) の比率は全数集計の真の値に一致する。実装の中核をリスト 1 に示す。なお NDL は、要求した LIMIT の値によらず 1 回の応答が最大 1000 行に無言で切り

詰められるサーバ側の制約を持つため、各層の取得はページング（OFFSET を進めながら要求件数に達するまで取得）で行っている。

```

1 # 全数集計で得たtop-kラベル値ごとの真の件数比でmax_nodesをサンプリングする
2 label_values = count_label_values(client, config.target_class,
3                                   config.label.property,
4                                   limit=config.label.top_k)
5 requested_per_class = _allocate_per_class(label_values,
6                                           config.sampling.max_nodes)
7
8 assigned = {} # uri -> label (頻度順で先着したものを保持)
9 for value_count in label_values: # 頻度降順
10     n = requested_per_class[value_count.value]
11     term = _value_term(value_count.value, value_count.value_type)
12     query_template = (
13         "SELECT ?s WHERE { "
14         f"?s a {iri(config.target_class)} . "
15         f"?s {iri(config.label.property)} {term} . "
16         "} LIMIT {limit} OFFSET {offset}") # 末尾は非f文字列
17 # NDLの結果 cap対策としてページングで要求件数nまで取得する
18 rows = client.paginate(query_template,
19                         page_size=config.sampling.page_size,
20                         max_total=n)
21 for row in rows:
22     assigned.setdefault(row["s"].value, value_count.value)

```

ソースコード 1: ラベル値ごとの層化抽出 (rdf2graph/convert/labels.py)。全数集計の真の比率でサンプリングし、層ごとにページングで取得する。

### 3.4 グラフ変換とラベルリークの防止

取得した1-hopトリプルから、edge\_type 付き単一の torch\_geometric.data.Data を構築する。ノードは分類対象クラスのインスタンスと、それらと1-hopで接続する周辺エンティティからなる。エッジはオブジェクトプロパティ（値がURI型のもの）をリレーションタイプとして保持し、各リレーションについて逆方向のリレーションを明示的に追加する（双方向メッセージパッシングのためのR-GCN標準前処理）。ノード特徴量は、構造特徴（次数とリレーションタイプ別の隣接数）に、ラベル・タイトル文字列の浅いテキスト特徴を連結したものである。テキストは文字  $n$ -gram (char-wb, 2-4) のハッシングベクトル化により128次元に落とす。文字  $n$ -gram を用いることで、分かち書きのない日本語も英語と同じコードのまま処理でき、言語ごとの分岐を持たずに済ませる。

ここで重要なのが、ラベルリークの防止である。ラベルの定義に用いた述語（NDLのskos:inScheme、DBLPのdblp:bibtexType）とrdf:typeを、エッジおよび特徴量の自動発見から無条件で除外する。これを残すと、モデルがグラフ構造からラベルをほぼ直接読み取ってしまう。特にDBLPでは、rdf:typeが持つサブタイプ（dblp:Inproceedingsなど）がbibtexTypeとほぼ同一の分類情報を別語彙で重複表現しているため、除外しなければ事実上ラベルを二重に与えることになる。この除外はR-GCN原論文がAIFB・MUTAG等の前処理で「ラベル定義に使った関係をグラフから除去する」と同じ標準的な慣行であり、場当たり的な措置ではないをここに記す。実装をリスト2に示す。

```

1 # ラベル定義に使ったlabel.propertyとrdf:typeを無条件除外(ラベルリーク防止)
2 exclude = {config.label.property, RDF_TYPE_URI}
3

```

```

4 def _select_edge_predicates(triples, total_target_nodes, exclude,
5                             min_coverage, max_properties):
6     subjects_by_predicate = defaultdict(set)
7     for subject, predicate, obj in triples:
8         if predicate in exclude or obj.type != "uri": # 除外述語とnon-
            URI値を弾く
9             continue
10        subjects_by_predicate[predicate].add(subject)
11        coverage = {p: len(s) / total_target_nodes
12                    for p, s in subjects_by_predicate.items()}
13        eligible = [p for p, c in coverage.items() if c >= min_coverage]
14        eligible.sort(key=lambda p: coverage[p], reverse=True)
15        return eligible[:max_properties]

```

ソースコード 2: ラベルリーク防止 (rdf2graph/convert/graph\_builder.py) 。  
label.property と rdf:type を除外し、被覆率でエッジ述語を選ぶ。

### 3.5 モデルとベースライン

R-GCN は、RGCNConv を 2 層重ね、その出力（埋め込み）を線形分類ヘッドに通す構成とした（リスト 3）。埋め込みと分類ロジットを明確に分離し、embed が返す最終分類層直前の出力を、後述するクラス分離度の算出に用いる。関係数が少ないエンドポイントでも num\_bases が関係数を超えないよう、基底数は関係数と上限値の小さい方に設定している。

これを 2 つのベースラインと比較する。第一に多数派クラスベースライン（学習集合で最頻のクラスを常に予測する）であり、精度の下限を与える。第二に特徴量のみベースラインであり、グラフ構造（エッジ）を一切用いず、ノード特徴量だけでロジスティック回帰を行う。後者は本実験の切り分けの要である。これがなければ、R-GCN の精度がグラフ構造に由来するのか、単にノード特徴量が強だけなのかを判別できない。両ベースラインは無条件で実装し、性能改善の工夫は加えない。

```

1 class RGCNClassifier(nn.Module):
2     def __init__(self, in_channels, hidden_channels,
3                 num_relations, num_classes, dropout=0.3):
4         super().__init__()
5         num_bases = min(num_relations, MAX_NUM_BASES)
6         self.conv1 = RGCNConv(in_channels, hidden_channels,
7                               num_relations=num_relations, num_bases=
8                               num_bases)
9         self.conv2 = RGCNConv(hidden_channels, hidden_channels,
10                               num_relations=num_relations, num_bases=
11                               num_bases)
12         self.classifier = nn.Linear(hidden_channels, num_classes)
13         self.dropout = dropout
14
15     def embed(self, x, edge_index, edge_type):
16         h = F.relu(self.conv1(x, edge_index, edge_type))
17         h = F.dropout(h, p=self.dropout, training=self.training)
18         h = F.relu(self.conv2(h, edge_index, edge_type))
19         return h # 分類ヘッド直前の埋め込み(クラス分離度の算出に使う)
20
21     def forward(self, x, edge_index, edge_type):
22         embeddings = self.embed(x, edge_index, edge_type)
23         logits = self.classifier(

```

```

22         F.dropout(embeddings, p=self.dropout, training=self.training))
23     return logits, embeddings

```

ソースコード 3: R-GCN 分類器 (rdf2graph/models/rgcn.py)。embed が分類ヘッド直前の埋め込みを返し、クラス分離度の算出に使う。

### 3.6 クラスセントロイド間コサイン類似度

本実験の主眼である比較指標を定義する。学習済み R-GCN が各分類対象ノード  $i$  に与える埋め込みを  $h_i \in \mathbb{R}^d$ 、そのラベルを  $y_i \in \{1, \dots, C\}$  とする。クラス  $c$  のセントロイド（平均ベクトル）を式 (1) で定める。ここで  $S_c = \{i : y_i = c\}$  である。

$$\mu_c = \frac{1}{|S_c|} \sum_{i \in S_c} h_i \quad (1)$$

クラスペア  $(c, c')$  間のコサイン類似度を  $s_{cc'} = \frac{\mu_c \cdot \mu_{c'}}{\|\mu_c\| \|\mu_{c'}\|}$  とし、対角を除く上三角の平均を、そのエンドポイントの分離度の要約統計量とする（式 (2)）。

$$\bar{s} = \frac{2}{C(C-1)} \sum_{c < c'} s_{cc'} \quad (2)$$

$\bar{s}$  が低いほど、クラスのセントロイドが埋め込み空間上で互いに離れており、そのデータセットのクラスはよく分離している。各エンドポイントは独立に学習し、ラベル空間も共有しないため、埋め込みベクトルそのものを横断比較することはできない。そこで本実験では、この要約統計量  $\bar{s}$  自体をエンドポイント間で比較する。評価指標は  $\bar{s}$  に加えて macro-F1・balanced accuracy・混同行列を用い、データ分割は train 0.7・val 0.15・test 0.15 の層化分割、乱数シードは 42 で固定した。

## 4. 結果と考察

数値は学習・評価スクリプトの出力 (RTX 5070 Ti・CUDA、シード 42) に基づく。構築されたグラフの規模は、NDL が分類対象 1 万 5 千ノード (1-hop 隣接を含む総ノード数 4 万 2545、エッジ 5 万 5112、リレーション 4 種)、DBLP が分類対象 1 万 5 千ノード (総ノード数 4 万 7149、エッジ 10 万〈上限による間引き後〉、リレーション 20 種) である。層化抽出後のラベル分布を図 1 に示す。NDL は最頻クラスの personalNames が 70.5% を占める強い不均衡を示し、DBLP は Article 51.0%・Inproceedings 45.4% の 2 クラスで大半を占める。

### 4.1 分類精度

モデル別の精度を表 2 に示す。最も重要な知見は**両エンドポイントとも、グラフ構造を一切用いない特徴量のみベースラインが R-GCN を上回った**ことである (NDL で 0.788 対 0.738、DBLP で 0.582 対 0.523)。これはベースラインを置いたことで得られた知見である。両ベースラインを併置したことで、次の切り分けが可能になった。多数派ベースライン (macro-F1 で NDL 0.166、DBLP 0.135) に対しては R-GCN・特徴量のみとも大きく上回るため、両データセットとも分類可能な信号は確かに存在する。しかしその信号はノード自身の特徴量、すなわちラベル・タイトル文字列の文字  $n$ -gram にほぼ尽きており、リレーション構造を加えて

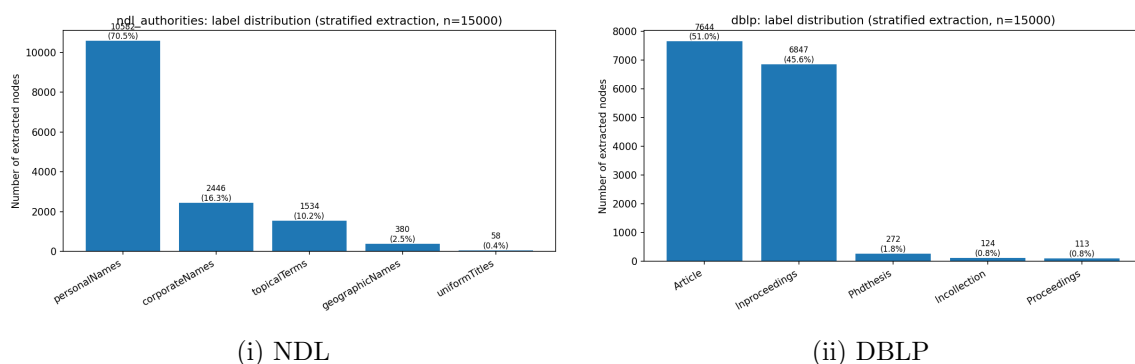


Figure 1: 層化抽出後のラベル分布。層間比率は全数集計の真の値に一致する。両データセットとも裾のクラスは極端に少ない。

Table 2: モデル別の分類精度 (test split、シード 42)。両データセットとも特徴量のみベースラインが R-GCN を上回った。

| データセット | モデル            | macro-F1      | balanced acc  | 学習時間 (s) |
|--------|----------------|---------------|---------------|----------|
| NDL    | R-GCN          | 0.7382        | 0.7490        | 1.77     |
| NDL    | 多数派            | 0.1655        | 0.2000        | 0.00     |
| NDL    | 特徴量のみ (LogReg) | <b>0.7881</b> | <b>0.7738</b> | 0.45     |
| DBLP   | R-GCN          | 0.5231        | 0.5214        | 3.56     |
| DBLP   | 多数派            | 0.1351        | 0.2000        | 0.00     |
| DBLP   | 特徴量のみ (LogReg) | <b>0.5821</b> | <b>0.5548</b> | 2.20     |

も精度は向上しない。むしろ R-GCN では近傍集約が局所的な文字列信号をわずかに希釈し、精度を下けている。

この所見はデータの性質と整合的である。NDL では典拠概念どうしがほとんど相互リンクを持たず、大半は VIAF や別エンティティといった外部ノードへ次数 1 で接続し、分類対象 1 万 5 千件のうち 1123 件は完全に孤立していた。「典拠の種別 (人名か地名か団体か)」は、名前の表層形にほぼ現れるため、リンク構造から追加で得られる情報は乏しい。DBLP でも「書誌タイプ (Article か Inproceedings か)」はタイトルや掲載情報の文字列パターンに強く現れる。すなわち、この 2 つの書誌ノード分類タスクでは、識別信号はノード自身のテキストに宿り、リンク構造には乏しいというのが、比較分析から得られた具体的な知見である。なお混同行列を確認すると、両データセットとも最小クラス (NDL の uniformTitles、DBLP の Phdthesis や Incollection) はほぼすべて多数側クラスへ誤分類されており、これはラベルが捉える裾クラスの極端な不均衡をそのまま反映している。R-GCN を勝たせるためのモデル調整 (層数やドロップアウトの探索) は、この所見を歪めるため意図的に行っていない。

## 4.2 クラス分離度の横断比較

本実験の主眼であるクラス分離度の比較を表 3 に、クラスペアごとのコサイン類似度ヒートマップを図 2 に示す。クラス間平均コサイン類似度は、NDL (0.384) の方が DBLP (0.457) より低く、NDL の方がクラスが埋め込み空間上でよく分離している。これは NDL の方が R-GCN



Table 3: クラス分離度の横断比較。 $\bar{s}$  (クラス間平均コサイン類似度) が低いほどよく分離している。

| データセット | クラス数 | クラス間平均コサイン類似度 $\bar{s}$ | R-GCN macro-F1 |
|--------|------|-------------------------|----------------|
| NDL    | 5    | <b>0.3837</b>           | 0.7382         |
| DBLP   | 5    | 0.4566                  | 0.5231         |

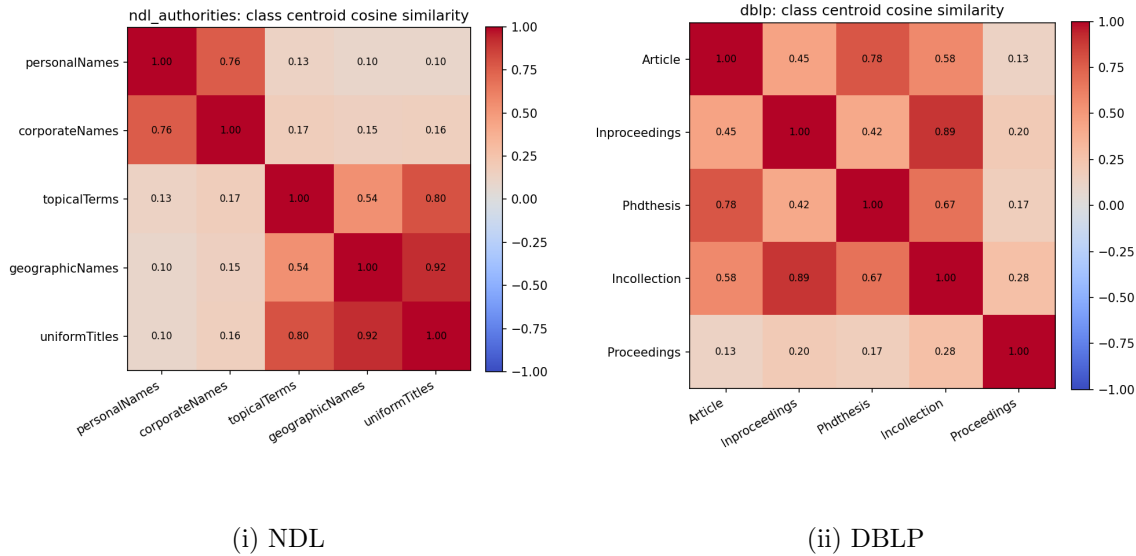


Figure 2: クラスセントロイド間コサイン類似度のヒートマップ。対角は1、値が小さい（青い）ほどクラスが離れている。

の macro-F1 が高い (0.738 対 0.523) ことと整合する。すなわち、埋め込み空間上のクラスセントロイド分離という内在的な指標が、実際の分類しやすさという外在的な指標と同じ向きを指しており、 $\bar{s}$  がデータセット横断の「クラスの分離しやすさ」の要約として機能していることを支持する。

ただし、この2値の単純な大小比較は慎重に扱う必要がある。両データセットはクラス数こそ同じ5であるが、不均衡度・言語・スキーマ・リレーション数 (NDL 4 種に対し DBLP 20 種) が大きく異なる。したがって「NDLの方が普遍的に分類しやすい」と一般化するのではなく、「本パイプラインのこの設定の下で、NDLの典拠種別クラスはDBLPの書誌タイプクラスより埋め込み空間上でよく分離した」という限定された主張にとどめる。

### 4.3 コスト

学習・推論はいずれも GPU 上で軽量であり、R-GCN の学習時間は NDL で 1.8 秒、DBLP で 3.6 秒、推論は 20 ミリ秒前後、学習時ピーク GPU メモリは NDL で約 250MB、DBLP で約 948MB であった。支配的なコストは学習ではなく SPARQL 抽出であり、1-hop プロパティの取得は NDL で約 58 秒、DBLP で約 1036 秒 (約 17 分) を要した。これは DBLP が 1 書誌あ

たりの述語数が多く、かつ GET の URL 長上限に収めるため対象 URI を 60 件ずつに分割して取得したことで往復回数が増えたためである。

## 5. 限界と今後の課題

1 つ目は、対象を 2 エンドポイントに縮小したことである。ICCU SBN と Cervantes Virtual は、ラベル候補述語の全数集計により追加のデータ品質問題が判明していた。ICCU SBN では件数が完全に一致する異表記のエイリアス Topic URI が多数存在し、上位 50 値を合わせても分類対象全体の 5.1% しか被覆しなかった。Cervantes Virtual では主題リテラルの最頻値が空文字列であり、被覆率も低かった。これらへの対応が提出期限までに完了しなかったため見送った。比較対象が 2 件に限られたことで、エンドポイント横断の一般化可能性は限定される。

2 つ目は、グラフ構造が精度に寄与しなかったことである。両データセットで特徴量のみベースラインが R-GCN を上回ったものは、書誌ノードのタイプ分類というタスク選択に対してリレーション構造の追加情報が乏しかったことを意味する。R-GCN の利点を引き出すには、ノード自身のテキストでは解けない別のラベル、例えば共著・引用ネットワークの構造に依存する研究分野や主題のカテゴリを分類対象に選ぶべきだった可能性がある。

3 つ目は、層化抽出は層間の比率を全数集計の真の値に一致させるものの、層内（同一ラベル値内）の抽出順序はサーバの既定順に依存しており、真の一樣ランダムではない。この層内順序バイアスは残存すると言える。これらは今後の課題である。

## 付録

本プロジェクトの実装（RDF → グラフ変換パイプラインおよび R-GCN ・ ベースラインの学習・評価コード）には、コーディング支援として Claude Code（使用モデル: Claude Opus 4.8）を用いた。

## References

- Michael Färber et al. AutoRDF2GML: Facilitating RDF integration in graph machine learning. In *The Semantic Web – ISWC 2024*. Springer, 2024.
- Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web – ESWC 2018*, pages 593–607. Springer, 2018.